Analyse

SAÉ 3.02 - Labyrinthe

Solution aux besoins

Étudiants :

Bredelle Victor
Marouze Antonin
Harlaut Romain
Zheng Angèl
Lavogiez Baptiste

Enseignant :

Mme Boneva

13 octobre 2025

Table des matières

1 Introduction

Le présent rapport détaille la composante `Analyse` relative au projet **SAÉ S3.02 - Labyrinthe**. Ce projet s'inscrit dans le cadre de la formation BUT Informatique et vise à développer les compétences en modélisation, conception orientée objet et travail collaboratif.

L'application développée propose un jeu de labyrinthe avec deux modes distincts : un **mode libre** permettant de jouer sur des labyrinthes générés aléatoirement, et un **mode progression** offrant une expérience structurée avec sauvegarde de l'avancement du joueur à travers différents niveaux de difficulté croissante.

Notre approche de modélisation repose sur les standards UML et suit une méthodologie itérative. Nous avons privilégié une démarche collaborative impliquant l'ensemble des membres de l'équipe, ainsi que des validations régulières auprès de nos enseignants spécialisés en analyse et en qualité de développement.

Le présent rapport aborde l'organisation interne de notre équipe ainsi que notre approche de modélisation des exigences du client, structurée comme suit :

1. Diagrammes de cas d'utilisation
2. Fiches descriptives
3. Prototype d'interface
4. Diagramme de classe
5. Traçabilité globale

Il est entendu que chaque section est itérative et a fait l'objet de plusieurs cycles de révision pour garantir la cohérence et la complétude de la modélisation.

2 Équipe et organisation du travail

Groupe G3.3

- Bredelle Victor
- Marouze Antonin
- Harlaut Romain
- Zheng Angèl
- Lavogiez Baptiste

> **1. Victor**

Rédaction de trois fiches descriptives avec Baptiste :

1. Charger profil
2. Afficher progression
3. Lancer mode progression

> **2. Antonin**

Réalisation de la maquette *Figma* avec Angèl et Romain. Contribution majeure sur l'expérience utilisateur et le design des écrans de progression.

> **3. Romain**

Réalisation de la fiche descriptive "Avancer d'une case", aide à la maquette *Figma*.

4. Angèl

Réalisation de la maquette *Figma* avec Antonin et Romain.

5. Baptiste

Rédaction de trois fiches descriptives avec Victor :

1. Charger profil
2. Afficher progression
3. Lancer mode progression

Rédaction du rapport.

Le groupe

Outre les responsabilités individuelles, nous avons collectivement débattu de la manière de modéliser chaque exigence en l'examinant sous divers angles. Ces discussions de groupe ont été essentielles pour converger vers des solutions consensuelles et optimales.

Information

L'ensemble des contributeurs a collaboré à l'élaboration du diagramme de classes.

3 Cas d'utilisation

3.1 Diagrammes de cas d'utilisation

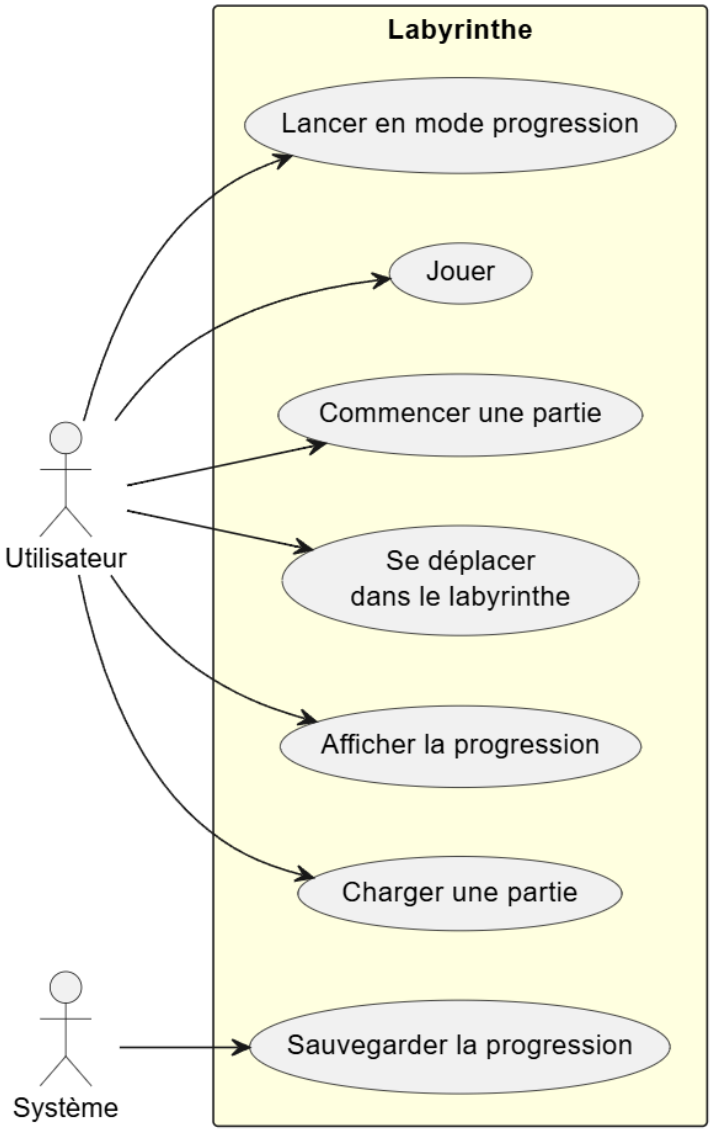

FIGURE 3.1 – Cas d'utilisation identifiés

3.1.1 Analyse du diagramme de cas d'utilisation

Le diagramme présenté ci-dessus modélise l'ensemble des interactions fonctionnelles du système Labyrinthe. Il identifie **sept cas d'utilisation principaux** répartis entre deux acteurs : l'Utilisateur et le Système.

3.2 Acteurs identifiés

- **Utilisateur** : acteur principal, représente le joueur interagissant avec l'application (lancement du jeu, déplacement dans le labyrinthe, consultation de la progression)
- **Système** : acteur secondaire représentant les processus automatiques de persistance des données (sauvegarde de la progression sans intervention explicite de l'utilisateur)

La présence d'un acteur "Système" se justifie dans notre contexte par la nécessité de modéliser explicitement les opérations de persistance automatique. Cette modélisation permet de distinguer clairement les actions intentionnelles de l'utilisateur des mécanismes de sauvegarde implicites déclenchés par le système après chaque action significative (complétion d'un niveau, modification du profil).

3.3 Cas d'utilisation

Le diagramme couvre intégralement le **mode progression**, qui constitue le cœur fonctionnel de l'application :

1. **Lancer en mode progression** : point d'entrée du mode progression, incluant la saisie du nom du joueur
2. **Charger une partie** : reprise d'une progression existante pour un joueur connu
3. **Jouer** : action générique d'interaction avec le jeu
4. **Commencer une partie** : démarrage d'une nouvelle session de jeu
5. **Se déplacer dans le labyrinthe** : déplacement case par case du joueur (cas d'utilisation détaillé dans la fiche "Déplacer joueur")
6. **Afficher la progression** : consultation du tableau de bord avec niveaux complétés, score, caractéristiques
7. **Sauvegarder la progression** : persistance automatique de l'état du joueur (acteur Système)

3.4 Clarifications

Le diagramme de cas d'utilisation reflète les choix de modélisation suivants :

- **Distinction entre "Charger une partie" et "Lancer en mode progression"** : Le premier cas concerne exclusivement la reprise d'un profil existant avec sa progres-

sion sauvegardée. Le second englobe l'initialisation du mode progression (saisie du nom, vérification d'existence, création ou chargement du profil).

- **Attribution de la sauvegarde au "Système"** : Conformément aux standards UML, les opérations automatiques de persistance sont modélisées par un acteur secondaire. Cette distinction clarifie que la sauvegarde intervient sans action explicite de l'utilisateur après chaque complétion de défi ou modification significative de l'état du joueur.
- **Hiérarchie "Jouer", "Commencer une partie" et "Se déplacer dans le labyrinthe"** : Le cas d'utilisation "Jouer" représente l'action globale d'interaction avec le jeu (démarrage, déplacement, consultation). "Commencer une partie" spécifie le démarrage effectif d'un défi après sélection dans l'interface de progression. "Se déplacer dans le labyrinthe" détaille l'action atomique de déplacement case par case, qui est l'interaction principale durant une partie en cours.
- **Mode libre** : Bien que non représenté dans ce diagramme centré sur le mode progression, le mode libre suit un flux simplifié : sélection du mode, paramétrage (taille, pourcentage de murs), génération du labyrinthe, et jeu sans sauvegarde de progression.

3.5 Fiches descriptives

3.5.1 Cas d'utilisation : Charger profil

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun

Préconditions : L'utilisateur est dans le menu de saisie du nom ("Lancer le mode progression")
Garanties en cas de succès : Le système charge le profil associé au nom et l'utilisateur reprend sa progression
Garanties minimales : L'intégrité des données existantes est préservée. Aucune donnée n'est corrompue

Scénario nominal :

1. L'utilisateur a rentré un nom déjà existant dans la base de données lors du **Cas d'utilisation** "Lancer mode progression" (suite de ce cas, ayant déjà vérifié l'occurrence)
2. Le système affiche un menu où l'utilisateur peut choisir de "Reprendre la partie existante" ou "Écraser la partie existante".
3. L'utilisateur sélectionne "Reprendre la partie"
4. Le système charge le profil associé au nom depuis la base de données et l'utilisateur reprend sa progression

Scénarios alternatifs : Aucun

> **Clarification**

Comme expliqué, cette fonctionnalité est la suite du Cas d'utilisation "Lancer mode progression" (Voir le scénario alternatif B de "Lancer mode progression").

3.5.2 Cas d'utilisation : Afficher progression

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun

Préconditions : L'utilisateur possède un profil et se trouve en mode progression.
Garanties en cas de succès : Le système affiche la progression associée au joueur (score, caractéristiques, etc.)
Garanties minimales : Les données affichées sont cohérentes. Le système ne modifie pas la progression

Scénario nominal :

1. L'utilisateur sélectionne la fonctionnalité "accéder au menu du mode progression"
2. Le système calcule le score et les caractéristiques du joueur. Puis, il affiche sa progression.

Scénarios alternatifs : Aucun

> **Clarification**

Conformément à la méthodologie dispensée en cours, la présente fiche détaille peu les décisions techniques, privilégiant une approche abstraite.

3.5.3 Cas d'utilisation : Lancer mode progression

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun

Préconditions : Le jeu a été lancé.
Garanties en cas de succès : L'utilisateur accède au mode progression
Garanties minimales : L'intégrité de la base de données est préservée. Aucune donnée n'est perdue

Scénario nominal :

1. L'utilisateur sélectionne la fonctionnalité "Lancer mode progression".
2. Le système affiche un menu où l'utilisateur peut saisir son nom
3. L'utilisateur saisit son nom
4. Le système vérifie que le nom est valide, sauvegarde son profil dans la base de données et lance le mode progression

Scénarios alternatifs :

A : Le système détecte une erreur lors de la vérification
4(A) : Le système affiche une notification (précisant la nature de l'erreur), puis revient en étape 2 du scénario principal

B : Le système détecte une occurrence du nom dans la base de données (charger profil)
4(B) : Le système démarre le **Cas d'utilisation** "Charger profil"

3.5.4 Cas d'utilisation : Déplacer joueur

Système : Labyrinthe
Acteur principal : Utilisateur
Déclencheur : Aucun
Autres acteurs : Aucun

Préconditions : Le joueur se trouve dans le labyrinthe.
Garanties en cas de succès : Le système déplace le joueur à la case souhaitée
Garanties minimales : La position du joueur reste cohérente. L'état du labyrinthe n'est pas corrompu

Scénario nominal :

1. Le joueur appuie sur une flèche directionnelle
2. Le système vérifie que la direction est accessible et déplace le joueur sur la case associée

Scénario alternatifs :

A : Le joueur atteint la sortie
2(A) : Le système affiche une notification annonçant que la sortie a été atteinte et redirige vers le menu principal.

B : Le joueur se déplace dans une case inaccessible
2(B) : Le système affiche la position actuelle et non la nouvelle position et revient en étape 1 du scénario principal

3.6 Prototypes d'interface

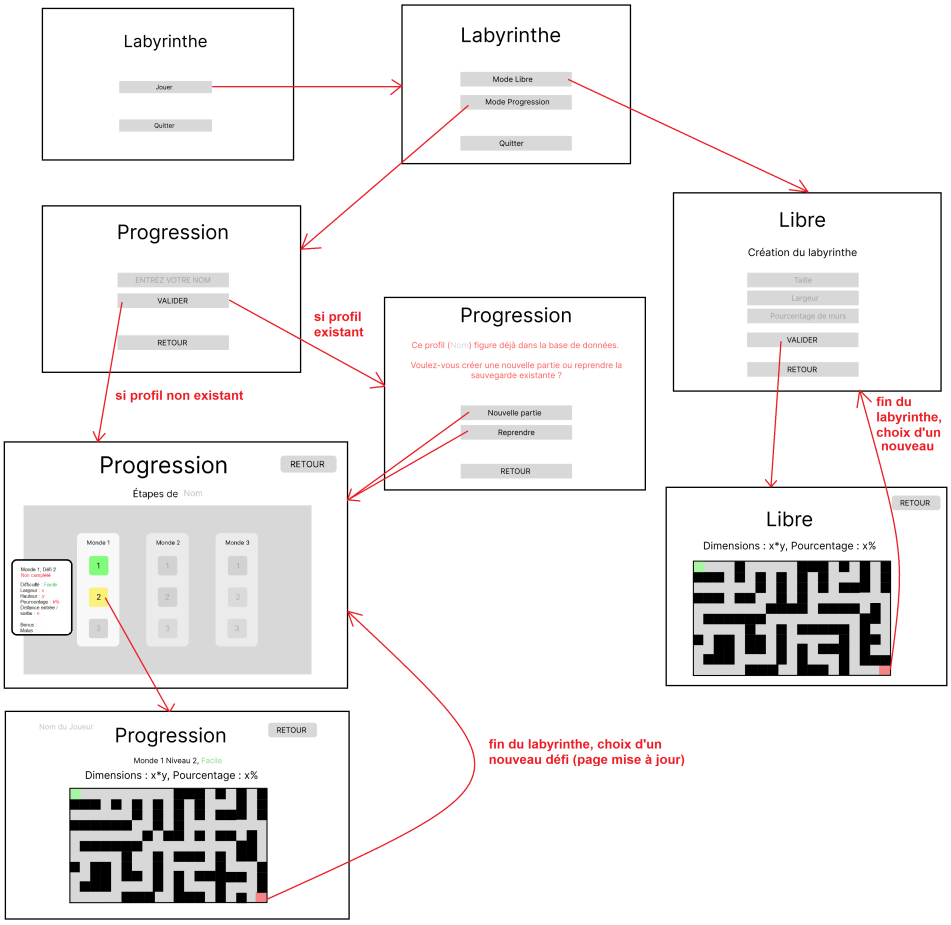

Figure 3.2 – Prototypes d'interface identifiés avec arborescence

3.6.1 Clarifications

Les boutons "Retour" et "Quitter" ont une flèche d'arborescence prévisible qui n'est pas mentionnée afin de ne pas surcharger le prototype d'interface.

La fenêtre de progression est bien entendu liée au profil chargé ; que l'on reprenne une partie ou que l'on en crée une nouvelle, la page de progression sera adaptée aux données de progression du joueur (monde, défis complétés...).

La fenêtre de progression comporte un élément spécifique : les informations du défi survolé (*hover*). Ce choix a été fait afin que le prototype comporte la visualisation de la progression ainsi que le choix du défi dans la même page ; deux pages différentes nécessiteraient plus d'arborescence (notamment un bouton visualiser progression...) résultant en moins de fluidité pour l'utilisateur. Lorsque l'on clique sur le défi, nous sommes donc emmenés

sur le labyrinthe correspondant au niveau (généré avec les caractéristiques de taille et de pourcentage du défi, conformément au sujet).

La voici en meilleure résolution :

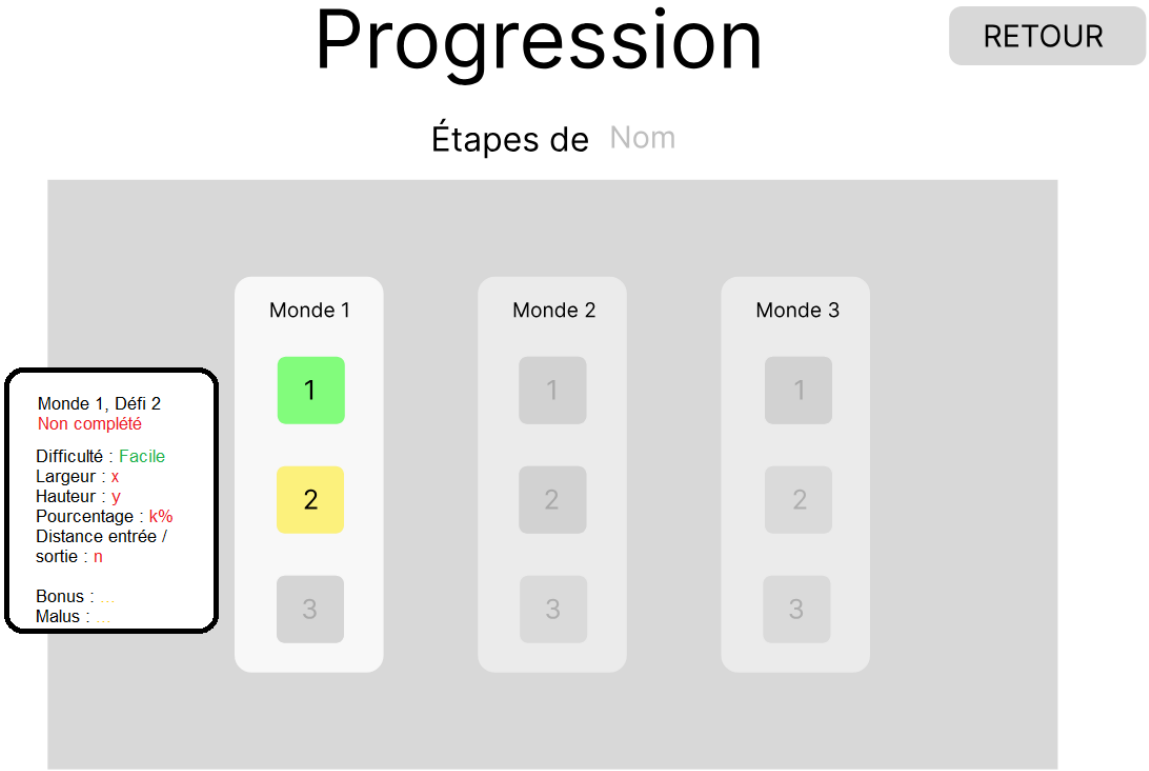

Figure 3.3 – Page de progression avec survol d'un niveau

Enfin, une fois la sortie d'un labyrinthe de mode progression atteinte, les données sont sauvegardées et la page de progression est donc mise à jour en conséquence.

3.6.2 Vue d'ensemble et parcours utilisateur

Le prototype présenté ci-dessus illustre le flux complet de navigation dans l'application pour les deux modes de jeu implémentés. Il matérialise visuellement les cas d'utilisation précédemment décrits et garantit une cohérence entre la modélisation fonctionnelle et l'expérience utilisateur finale.

Le prototype couvre **huit écrans distincts** répartis entre deux parcours utilisateur : le mode progression et le mode libre. Ces parcours partagent un point d'entrée commun (le menu principal) avant de diverger selon le mode choisi par l'utilisateur.

Parcours utilisateur – Mode Progression (6 écrans) :

1. **Menu principal** : point d'entrée de l'application avec les options "Jouer" et "Quitter"
2. **Choix du mode** : sélection entre "Mode Libre" et "Mode Progression"
3. **Saisie du nom** : identification ou création du profil joueur
4. **Gestion du profil existant** : choix entre "Nouvelle partie" et "Reprendre" si un profil existe
5. **Sélection monde/niveau** : tableau de progression organisé en trois mondes avec niveaux numérotés, utilisant un code couleur pour indiquer l'état de complétion / accès. L'utilisateur doit cliquer pour sélectionner son défi.
6. **Interface de jeu** : labyrinthe avec déplacement du joueur (carré vert) vers la sortie (carré rouge)

Parcours utilisateur – Mode Libre (4 écrans) :

1. **Menu principal** : point d'entrée de l'application
2. **Choix du mode** : sélection entre "Mode Libre" et "Mode Progression"
3. **Configuration du labyrinthe** : paramétrage des dimensions (taille, largeur) et du pourcentage de murs
4. **Interface de jeu** : labyrinthe généré aléatoirement selon les paramètres choisis

Ces deux séquences garantissent une progression logique et intuitive adaptée aux besoins spécifiques de chaque mode, sans étape superflue, dans une stricte conformité aux exigences du client.

3.6.3 Correspondance avec les cas d'utilisation

Le tableau suivant établit la correspondance entre les écrans du prototype (mode progression uniquement) et les cas d'utilisation modélisés :

Cas d'utilisation	**Écrans concernés**
Lancer mode progression	Menu principal → Choix du mode → Saisie du nom
Charger profil	Popup de profil existant (choix "Reprendre" ou "Nouvelle partie")
Jouer	Sélection niveau → Labyrinthe en jeu
Commencer une partie	Sélection niveau (clic sur défi) → Labyrinthe en jeu
Afficher progression	Sélection niveau (affichage des 3 mondes avec code couleur)
Se déplacer dans le labyrinthe	Labyrinthe en jeu (déplacement carré vert → carré rouge)
Sauvegarder progression	Automatique après sortie (mise à jour de la sélection niveau)

Cette traçabilité garantit que l'ensemble des fonctionnalités modélisées du mode progression dispose d'une représentation visuelle concrète, et inversement, qu'aucun élément de l'interface n'est superflu par rapport aux exigences fonctionnelles.

4 Diagramme de classe

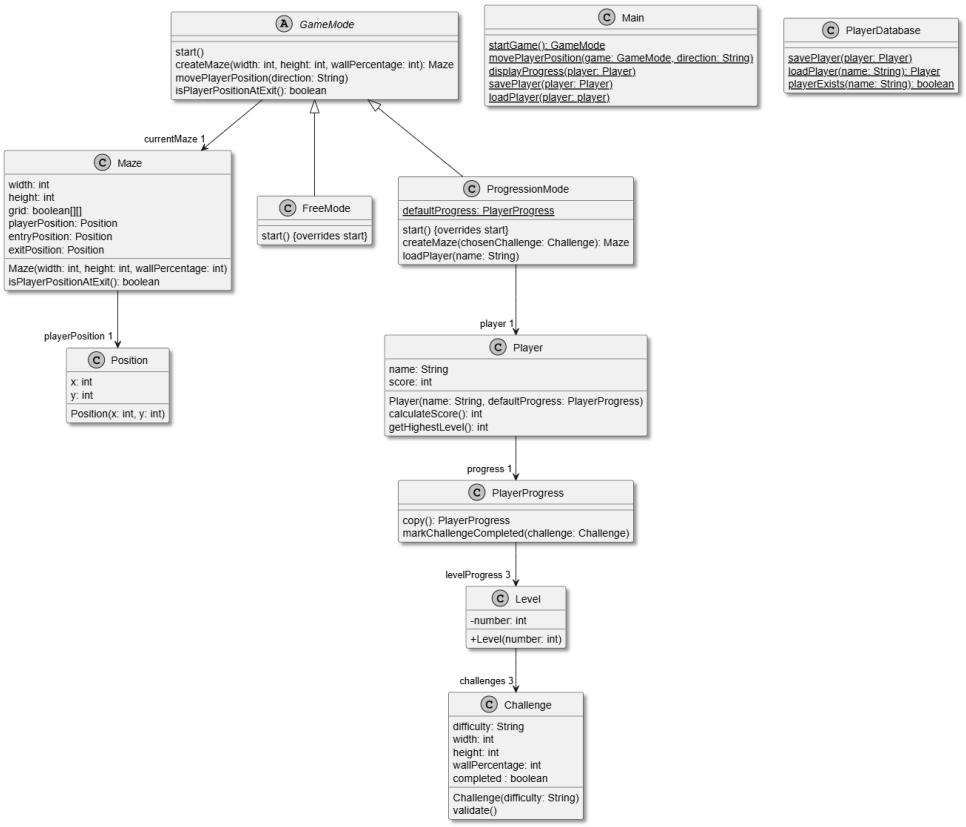

Figure 4.1 – Diagramme de classe identifié

Le diagramme de classe a été généré à l'aide de *PlantUML* dans un format textuel, assurant ainsi une portabilité et une facilité de modification accrues.

La conception de ce diagramme a nécessité un investissement conséquent, considérant que, par nature, un diagramme de classe requiert une **réflexion soutenue et l'intégration de multiples perspectives**.

L'approche collaborative adoptée par l'équipe a permis d'aboutir à une solution cohérente et robuste.

Des retours ont été sollicités auprès des professeurs spécialisés en analyse, ainsi qu'auprès de ceux enseignant la qualité de développement, garantissant la conformité de notre modélisation aux standards académiques.

4.1 Structure générale

L'architecture se décompose en trois niveaux conceptuels :

1. **Niveau contrôleur** : `Main` orchestre le flux général (lancement du jeu, sélection du mode, interaction avec la base de données)
2. **Niveau logique métier** : `GameMode`, `Maze`, `Player`, `PlayerProgress`, `Level`, `Challenge` encapsulent les règles métier et l'état de la partie
3. **Niveau persistance** : `PlayerDatabase` isole les opérations de sauvegarde et de chargement

Cette séparation en couches garantit une maintenance facilitée et une testabilité accrue : chaque niveau peut être testé indépendamment, réduisant ainsi les risques d'anomalies et facilitant l'évolution future du système.

4.2 Conformité avec la méthode vue en cours

La méthodologie vue en `Analyse` préconise l'ordre suivant :

1. Une classe `Main` est créée
2. Elle comporte une fonction par cas d'utilisation
3. Pour chaque cas d'utilisation, on enrichit le diagramme de façon à la supporter

Notre conception respecte cette méthodologie tout en l'adaptant aux spécificités du projet.

4.3 Correspondance avec les cas d'utilisation

Afin de démontrer que l'ensemble des cas d'utilisation sont effectivement couverts par la classe `Main`, nous établissons la correspondance suivante :

Fonctions du `Main`	**Cas d'utilisation correspondants**
`startGame(): GameMode`	Lancer mode progression + Commencer une partie (englobe donc Jouer)
`movePlayerPosition(game: GameMode, direction: String)`	Se déplacer dans le labyrinthe
`displayProgress(player: Player)`	Afficher progression
`savePlayer(player: Player)`	Sauvegarder progression (donc le joueur car contenue dans le joueur)
`loadPlayer(name: String): Player`	Charger profil

Cette approche évite la création de fonctions redondantes qui résulteraient d'une correspondance stricte entre chaque cas d'utilisation et chaque méthode. En effet, plusieurs cas d'utilisation partagent une logique commune et bénéficient d'un regroupement fonctionnel.

4.4 Équilibre entre méthodologie et besoin client

Notre modélisation représente un équilibre optimal entre le respect de la méthodologie d'`Analyse` et la satisfaction des exigences fonctionnelles exprimées dans le sujet.

En effet, une application strictement littérale de la méthodologie (une fonction par cas d'utilisation) aurait généré des fonctions superflues ou redondantes, notamment pour "Jouer" et "Commencer une partie", compte tenu de la structure actuelle de nos classes.

À l'inverse, ignorer complètement la méthodologie aurait compromis la cohérence pédagogique du projet.

Notre solution concilie ces deux impératifs : les cas d'utilisation présentant un recouvrement fonctionnel ("Jouer", "Commencer une partie", "Lancer mode progression") sont unifiés au sein de la méthode `startGame(): GameMode`.

L'utilisation d'un type abstrait `GameMode` et du polymorphisme permet de préserver la distinction nécessaire entre les différents modes de jeu tout en mutualisant le code commun.

4.5 Clarifications

La fonction `startGame()`, comme indiqué dans le tableau précédent, supporte plusieurs cas d'utilisation. Ce choix de conception mérite une explication détaillée :

Cette méthode retourne un `GameMode`, qui peut aussi bien être le mode libre que le mode progression (type abstrait hérité). Elle est conçue pour être appelée depuis le menu de départ.

Ce type abstrait permet un traitement unifié des deux modes de jeu pour les opérations communes (par exemple, `movePlayerPosition`), évitant ainsi les duplications de code.

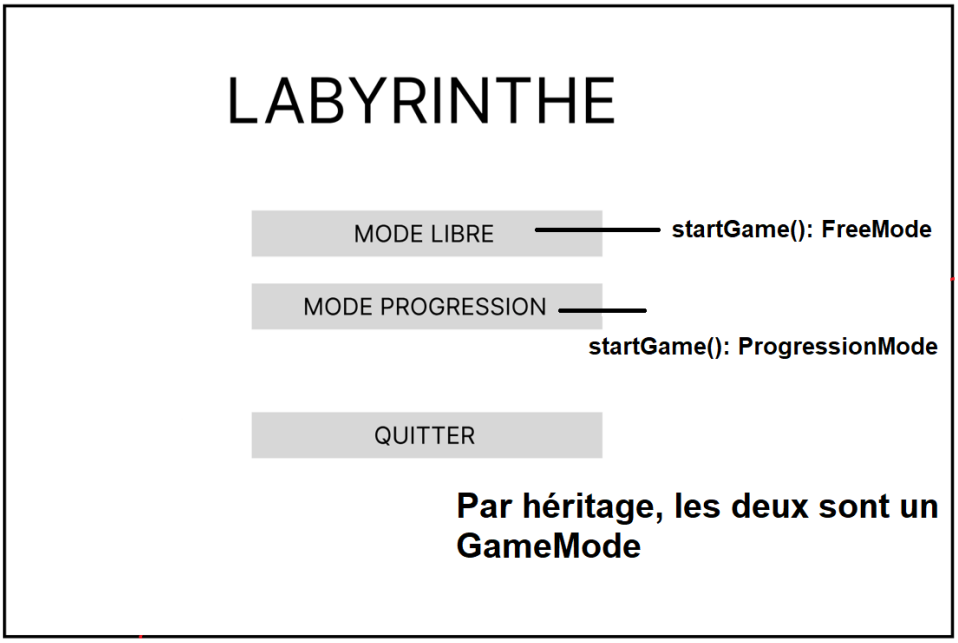

Figure 4.2 – startGame() dans le Menu

Elle sert donc à démarrer une partie, quel que soit le mode choisi, et couvre naturellement le cas d'utilisation "Lancer mode progression".

Dans le cas d'un lancement en mode progression, le joueur est dirigé vers une interface de saisie du nom. Une fois le nom validé, la fonction

`loadPlayer(name: String): Player` est invoquée afin de :

- Charger le profil existant si un joueur avec ce nom est enregistré dans la base de données, permettant ainsi de reprendre la progression sauvegardée
- Créer un nouveau profil si aucun joueur avec ce nom n'existe, assurant une expérience fluide pour les nouveaux joueurs

Cette conception garantit une gestion cohérente et efficace du cycle de vie des profils joueurs.

5 Traçabilité globale

La cohérence de notre modélisation repose sur une traçabilité bidirectionnelle entre les différents artefacts produits. Cette section synthétise les correspondances établies tout au long du rapport.

5.1 Traçabilité verticale : Du sujet au code

La chaîne de traçabilité suivante garantit que chaque exigence du sujet se retrouve dans notre conception :

1. **Exigences du sujet** : Les deux modes (libre/progression), la structure des défis, la sauvegarde de progression
2. **Cas d'utilisation** : Sept cas identifiés couvrant l'ensemble des interactions utilisateur-système
3. **Prototypes d'interface** : Huit écrans représentant visuellement les cas d'utilisation (tableaux de correspondance lignes 298-317)
4. **Diagramme de classe** : Classes et méthodes implémentant les cas d'utilisation (tableau de correspondance lignes 372-386)

5.2 Traçabilité horizontale : Cohérence inter-artefacts

Les tableaux de correspondance établis dans ce rapport démontrent la cohérence entre :

- **Cas d'utilisation ↔ Écrans** : Chaque cas d'utilisation du mode progression est représenté par un ou plusieurs écrans du prototype (section 3.5.3)
- **Cas d'utilisation ↔ Méthodes de Main** : Chaque cas d'utilisation est implémenté par une ou plusieurs méthodes de la classe `Main` (section 4.3)
- **Fiches descriptives ↔ Diagramme de classe** : Les scénarios décrits dans les fiches se traduisent par des appels de méthodes cohérents avec l'architecture proposée

Cette double traçabilité garantit l'absence d'incohérences ou d'éléments orphelins dans notre conception, facilitant ainsi la phase d'implémentation et les évolutions futures du système.

6 Conclusion

Le présent rapport a exposé l'ensemble de notre démarche de modélisation pour le projet SAÉ S3.02 - Labyrinthe. Nous avons couvert les sept cas d'utilisation identifiés, développé quatre fiches descriptives détaillées, conçu huit écrans de prototype d'interface, et élaboré un diagramme de classe structuré en trois couches (contrôleur, logique métier, persistance).

Notre approche a privilégié la conformité avec la méthodologie enseignée en cours d'Analyse, tout en l'adaptant aux spécificités fonctionnelles du sujet. La structure en mondes et défis du mode progression a été fidèlement modélisée, depuis les cas d'utilisation jusqu'aux classes `Challenge` et `Level`, en passant par les prototypes d'interface avec leur système de code couleur et de sélection par clic.

Les choix de conception (acteur Système pour la sauvegarde automatique, type abstrait `GameMode` pour mutualiser le code entre modes libre et progression, séparation en couches architecturales) ont été justifiés par des considérations de maintenabilité, d'extensibilité et de testabilité. Ces principes garantissent que le code produit sera robuste et évolutif, notamment pour l'intégration future des étapes supplémentaires du jalon 2.

La traçabilité globale établie tout au long du rapport (tableaux de correspondance, clarifications, cohérence inter-artefacts) assure que chaque exigence du sujet se retrouve dans notre conception, sans éléments superflus ni manquants.

L'ensemble de l'équipe a contribué à cette modélisation à travers des discussions collectives, une répartition claire des responsabilités (fiches descriptives, maquette Figma, diagramme de classe), et des validations régulières auprès des enseignants spécialisés. Cette démarche collaborative a permis de converger vers des solutions consensuelles et optimales, en examinant chaque exigence sous divers angles.

Ce travail constitue la base de la phase d'implémentation du jalon 1, qui pourra s'appuyer sur des spécifications fonctionnelles et techniques rigoureuses, complètes et cohérentes.

Merci d'avoir consulté cette présentation